TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

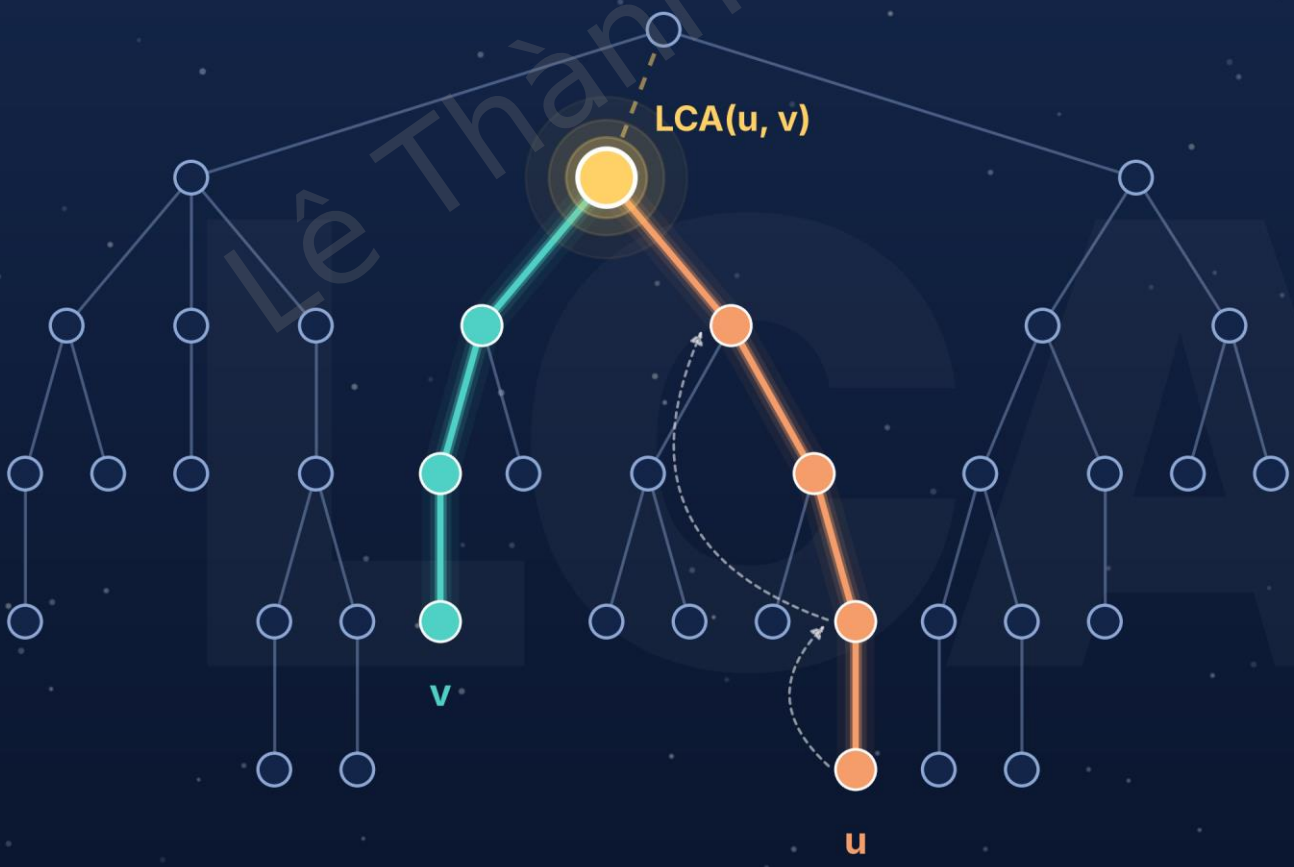
TỔ TIÊN CHUNG GẦN NHẤT

Lowest Common Ancestor

Từ cơ bản đến nâng cao

TẬP 1 LÝ THUYẾT

```
up[k][u] = up[k-1][up[k-1][u]]
```



Lê Thành Nam

Tổng hợp và biên soạn

Lê Thành Nam

1 Lý thuyết 2 Bài tập 3 Hướng dẫn giải

LỜI NÓI ĐẦU

Trong các đề thi học sinh giỏi Tin học, bài toán trên cây xuất hiện rất thường xuyên, và ở phần lớn những bài ấy, nhân vật chính là **tổ tiên chung gần nhất** (LCA, Lowest Common Ancestor). Có khi đề bài hỏi thẳng "tìm LCA của u và v". Có khi LCA ẩn mình sau một câu hỏi tưởng chừng không liên quan: Khoảng cách giữa hai thành phố, cạnh có tải trọng nhỏ nhất trên tuyến đường, số lần mỗi con đường được đi qua, hay tổng độ dài nhỏ nhất để nối một nhóm điểm. Nắm chắc LCA là nắm trong tay chiếc chìa khóa mở rất nhiều cánh cửa.

Bộ tài liệu này được viết để giúp em đi từ con số không tới mức sử dụng thành thạo LCA trong những bài toán khó. Bộ sách gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** (cuốn sách này) gồm 12 bài học chia thành 4 chương: Nhập môn tổ tiên chung gần nhất; Ba phương pháp tìm LCA; Ứng dụng của LCA; LCA nâng cao.
- **Tập 2: Bài tập** gồm 36 bài, mỗi bài học có ba bài tập ở ba mức độ (cơ bản, trung bình, nâng cao), có ràng buộc theo subtask như đề thi thật.
- **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm của từng cách làm.

Mỗi bài học không đưa ra thuật toán ngay. Em sẽ được đặt trước một bài toán cụ thể, thử cách ngây thơ, đếm số phép tính để thấy chỗ bế tắc, rồi được dẫn từng bước để phá bế tắc. Mọi ví dụ đều được chạy tay trên hình vẽ, và cả cuốn sách dùng chung một cây gia phả 14 người để em quen dần với từng đỉnh, từng nhánh. Mọi chương trình trong sách đều đã được kiểm tra bằng cách đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên, gồm cả những cây "xấu" như đường thẳng dài hay hình sao.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến. Mọi chương trình duyệt cây đều không dùng đệ quy, để không bị tràn ngăn xếp trên những cây sâu hàng trăm nghìn tầng. Tên biến được đặt ngắn gọn và thống nhất với kí hiệu trong công thức (p, dep, up, tin, tout, ...), để em đọc công thức và đọc chương trình như đọc cùng một thứ.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường, giống như cách một người giỏi thật sự suy nghĩ khi gặp bài mới:

Phần	Nội dung
Khởi động	Một câu hỏi nhỏ, gần gũi để em bắt đầu nghĩ.
Bài toán	Đề bài cụ thể, có ví dụ và hình minh họa.
Ý tưởng ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra, kèm sơ đồ khối và phân đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm, và đặt câu hỏi dẫn tới ý tưởng mới.
Phá bế tắc	Từng bước: Quan sát, công thức, chạy tay, chứng minh, chương trình.
Mở rộng	Các biến thể thường gặp trong đề thi.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
ĐỊNH NGHĨA	Khái niệm mới, định nghĩa chính xác.
CÔNG THỨC	Công thức then chốt của bài.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp, giải thích vì sao là O(h), O(log n), O(1), ...
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Lập luận chứng minh tính đúng đắn.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.
ĐỌC THÊM	Nội dung mở rộng dành cho học sinh muốn tìm hiểu sâu hơn.

Cây ví dụ dùng chung. Hầu hết các hình trong sách vẽ trên cùng một cây 14 đỉnh, gốc 1, với mảng cha p[2..14] = 1 1 1 2 2 3 4 4 6 6 7 9 10. Hãy chép cây này ra một tờ giấy riêng và để bên cạnh khi học.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Mỗi khi gặp một bảng chạy tay, hãy che phần đáp án lại và tự điền trước. Mỗi khi gặp một thuật toán, hãy tự trả lời ba câu hỏi: Chuẩn bị gì trước (tiền xử lý), mỗi câu hỏi làm gì, và vì sao đủ nhanh. Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

CHƯƠNG 1. NHẬP MÔN TỔ TIÊN CHUNG GẦN NHẤT **5**
Bài 1. Tổ tiên chung gần nhất và cách leo cây 6
Bài 2. Duỗi cây: Euler tour, tin và tout 22

CHƯƠNG 2. BA PHƯƠNG PHÁP TÌM LCA **33**
Bài 3. Nhảy nhị phân: Tổ tiên thứ k và LCA 34
Bài 4. Euler tour và bảng thưa: LCA trong O(1) 46
Bài 5. Thuật toán Tarjan: Đọc trước mọi câu hỏi 56

CHƯƠNG 3. ỨNG DỤNG CỦA LCA **66**
Bài 6. Khoảng cách và đỉnh thứ k trên đường đi 67
Bài 7. Giá trị lớn nhất, nhỏ nhất trên đường đi và cây khung 75
Bài 8. Cộng dồn trên cây: Hiệu tiền tố và tổng trên đường đi 86

CHƯƠNG 4. LCA NÂNG CAO **95**
Bài 9. Chia nặng nhẹ (HLD): LCA với bộ nhớ O(n) 96
Bài 10. Khi gốc thay đổi và vị trí tương đối của đường đi 106
Bài 11. Cây ảo: Chỉ giữ lại những đỉnh quan trọng 115
Bài 12. Cây lớn dần và đường kính động 124

Lê Thành Nam

CHƯƠNG 1. NHẬP MÔN TỔ TIÊN CHUNG GẦN NHẤT

Mỗi dòng họ Việt Nam đều có một cuốn gia phả. Hai người cùng họ gặp nhau lần đầu thường hỏi: "Chúng ta chung ông nào?" Người ông chung ấy, người gần nhất về phía trên mà cả hai cùng là con cháu, chính là tổ tiên chung gần nhất. Trong Tin học, câu hỏi ấy được viết thành LCA(u, v), và nó xuất hiện ở khắp nơi: Tính khoảng cách giữa hai thành phố trong một mạng đường không có vòng, tìm cạnh yếu nhất trên một tuyến vận chuyển, đếm số lần một con đường được đi qua.

Chương này đặt nền móng cho cả bộ sách. Em sẽ hiểu thật kĩ LCA là gì, tự tay viết cách tìm LCA đơn giản nhất bằng cách "leo cây", rồi học cách "duỗi" cây thành một dãy số để trả lời câu hỏi "u có phải tổ tiên của v không" chỉ bằng hai phép so sánh. Mọi phương pháp nhanh ở các chương sau đều được xây trên hai bài học này.

TRONG CHƯƠNG NÀY

Bài 1. Tổ tiên chung gần nhất và cách leo cây
Bài 2. Duỗi cây: Euler tour, tin và tout

Lê Thành Nam

BÀI 1 Tổ tiên chung gần nhất và cách leo cây

Học xong bài này, em sẽ:

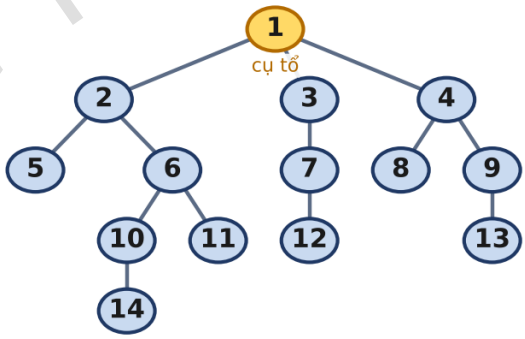
- ✓ Nắm vững các khái niệm cây có gốc, cha, con, tổ tiên, độ sâu và định nghĩa chính xác của LCA.
- ✓ Biết các tính chất quan trọng nhất của LCA và vì sao chúng đúng.
- ✓ Thấy tận mắt vì sao cách "liệt kê rồi so từng cặp" chậm, và đếm được số phép tính của nó.
- ✓ Cài đặt được cách "cân độ sâu rồi cùng leo" tìm LCA trong O(h), với h là chiều cao cây.
- ✓ Dùng LCA để tính khoảng cách giữa hai đỉnh và nhận ra chỗ bế tắc còn lại khi cây rất cao.

KHỞI ĐỘNG

Trong buổi họp mặt dòng họ, An (người số 14) và Bình (người số 11) lần đầu gặp nhau. Cả hai đều mang theo một trang gia phả: An biết bố mình là người số 10, ông nội là người số 6; Bình biết bố mình là người số 6. Vậy hai bạn có quan hệ gì? Người tổ tiên chung gần nhất của hai bạn là ai?

Với gia phả nhỏ trong Hình 1.1, em có thể dò bằng mắt. Nhưng nếu cuốn gia phả ghi hai trăm nghìn người và có năm mươi nghìn cặp người cần hỏi thì sao? Bài học này sẽ giúp em viết một chương trình trả lời toàn bộ các câu hỏi ấy trong chưa đầy nửa giây.

Lê Thanh Nam



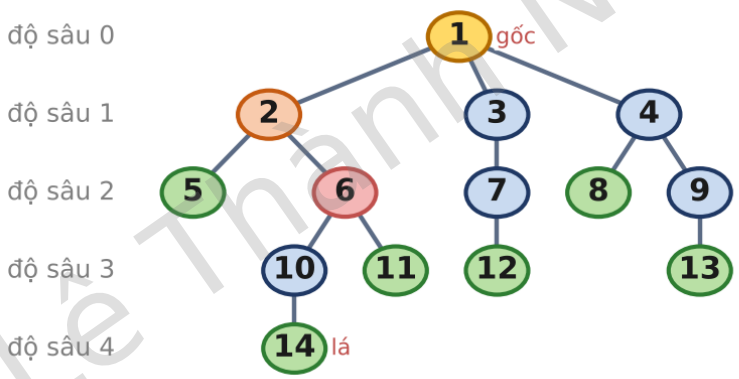
Hình 1.1. *Gia phả của một dòng họ gồm 14 người. Người số 1 (tô vàng) là cụ tổ. Cây này được dùng làm ví dụ xuyên suốt cuốn sách.*

1. Cây có gốc và các thuật ngữ

Một **cây** là một đồ thị vô hướng liên thông và không có chu trình. Cây n đỉnh luôn có đúng n − 1 cạnh, và giữa hai đỉnh bất kì có **đúng một đường đi đơn** (đường đi không lặp lại đỉnh). Tính chất cuối cùng là nền tảng của mọi thứ trong cuốn sách: Vì đường đi giữa hai đỉnh là duy nhất, ta mới có thể nói "đường đi từ u tới v" mà không sợ nhầm lẫn.

Khi chọn một đỉnh làm **gốc** và "treo" cả cây từ gốc xuống, ta được một **cây có gốc** như Hình 1.2. Ta dùng các thuật ngữ sau.

Thuật ngữ	Ý nghĩa (ví dụ trên Hình 1.2)
Gốc	Đỉnh được chọn làm đỉnh cao nhất. Ở đây là đỉnh 1.
Cha p[u]	Đỉnh liền trên u trên đường từ u về gốc. p[6] = 2. Gốc không có cha, quy ước p[gốc] = 0.
Con	Các đỉnh nhận u làm cha. Đỉnh 6 có hai con là 10 và 11.
Độ sâu dep[u]	Số cạnh trên đường từ gốc tới u. dep[1] = 0, dep[6] = 2, dep[14] = 4.
Chiều cao h	Độ sâu lớn nhất trong cây. Cây ví dụ có h = 4.
Tổ tiên của u	Các đỉnh nằm trên đường từ u về gốc, **kể cả chính u**. Tổ tiên của 14 là 14, 10, 6, 2, 1.
Hậu duệ của u	Các đỉnh nhận u làm tổ tiên. Hậu duệ của 6 là 6, 10, 11, 14.
Lá	Đỉnh không có con: 5, 8, 11, 12, 13, 14.
Cây con gốc u	Tập hợp mọi hậu duệ của u, cùng các cạnh giữa chúng.



Hình 1.2. Cây gốc 1 với độ sâu ghi bên trái. Đỉnh 2 (cam) là cha của đỉnh 6 (đỏ); các lá tô xanh lá.

QUY ƯỚC QUAN TRỌNG

Trong cả bộ sách, **mỗi đỉnh được coi là tổ tiên của chính nó**. Khi cần loại trừ chính u, ta nói "tổ tiên thực sự". Quy ước này làm cho mọi công thức gọn hơn rất nhiều: Chẳng hạn LCA(u, u) = u, và nếu u là tổ tiên của v thì LCA(u, v) = u.

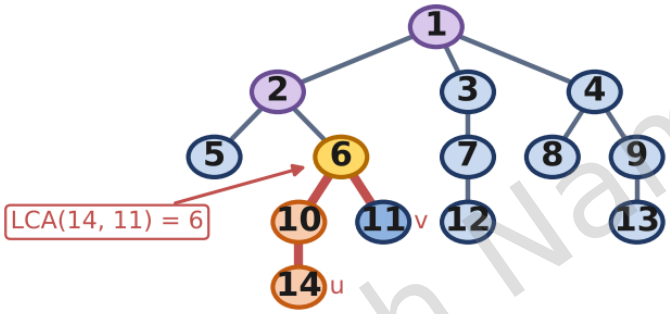
Lê Thành Nam

2. Tổ tiên chung gần nhất là gì?

ĐỊNH NGHĨA

Cho cây có gốc và hai đỉnh u, v. Một đỉnh w là **tổ tiên chung** của u và v nếu w vừa là tổ tiên của u, vừa là tổ tiên của v. Trong các tổ tiên chung, đỉnh có **độ sâu lớn nhất** (xa gốc nhất, gần u và v nhất) được gọi là **tổ tiên chung gần nhất** của u và v, kí hiệu **LCA(u, v)** (viết tắt của Lowest Common Ancestor).

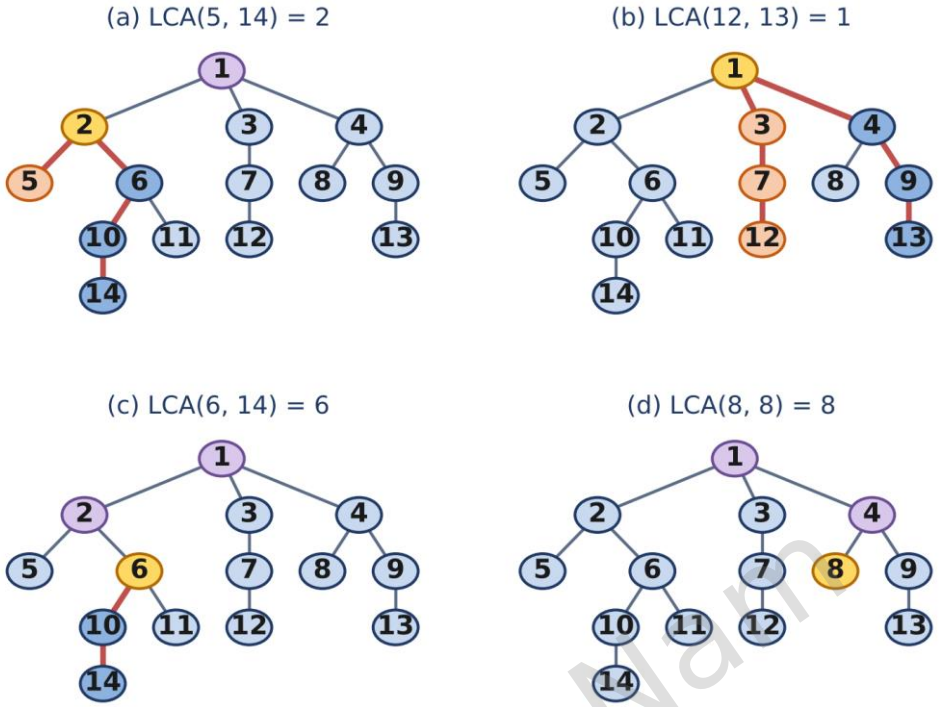
Gốc luôn là tổ tiên chung của mọi cặp đỉnh, nên tập tổ tiên chung không bao giờ rỗng. Ta sẽ thấy ngay dưới đây rằng đỉnh sâu nhất trong tập ấy là duy nhất.



Hình 1.3. *Tổ tiên của u = 14 tô cam, tổ tiên của v = 11 tô xanh, tổ tiên chung tô tím. Tổ tiên chung sâu nhất là 6 (tô vàng). Đường đi từ 14 tới 11 (tô đỏ) đi qua đúng đỉnh 6.*

Nhìn Hình 1.3, ta thấy tổ tiên của 14 là {14, 10, 6, 2, 1}, tổ tiên của 11 là {11, 6, 2, 1}. Phần chung là {6, 2, 1}. Ba đỉnh này nằm trên **một đường thẳng** từ 6 lên gốc; đỉnh sâu nhất là 6. Vậy LCA(14, 11) = 6, và An với Bình là chú cháu họ: Ông nội của An (người 6) chính là bố của Bình.

Lê Thanh Nam



Hình 1.4. *Bốn ví dụ. Hình (c): 6 là tổ tiên của 14 nên LCA(6, 14) = 6. Hình (d): LCA của một đỉnh với chính nó là chính nó.*

Lê Thành Nam

Các tính chất của LCA

Những tính chất sau sẽ được dùng đi dùng lại trong cả bộ sách. Hãy đọc chậm và tự kiểm tra từng tính chất trên cây ví dụ.

1. **Tập tổ tiên chung là một "đoạn thẳng" từ LCA lên gốc.** Tổ tiên của u xếp thành một dãy u, p[u], p[p[u]], ..., gốc. Nếu w là tổ tiên chung thì mọi tổ tiên của w cũng là tổ tiên chung. Do đó tổ tiên chung sâu nhất là duy nhất.

2. **LCA(u, v) = u khi và chỉ khi u là tổ tiên của v.** Đặc biệt LCA(u, u) = u.

3. **Đường đi từ u tới v đi qua LCA(u, v)**, và có dạng: Đi lên từ u tới LCA, rồi đi xuống từ LCA tới v. LCA là đỉnh có độ sâu nhỏ nhất trên đường đi đó.

4. **Tính đối xứng và kết hợp:** LCA(u, v) = LCA(v, u) và LCA(u, v, w) = LCA(LCA(u, v), w). Nhờ đó tìm được LCA của cả một nhóm đỉnh bằng cách gộp dần từng đỉnh.

5. **LCA phụ thuộc vào gốc.** Cùng một cây, đổi gốc thì LCA thay đổi. Chẳng hạn nếu chọn gốc là 14 thì LCA(5, 11) = 6 chứ không còn là 2. Bài 10 sẽ học cách xử lý khi gốc thay đổi.

VÌ SAO ĐÚNG?: VÌ SAO ĐƯỜNG ĐI TỪ U TỚI V ĐI QUA LCA?

Gọi l = LCA(u, v). Đi từ u lên l theo các cạnh cha, rồi từ l xuống v, ta được một đường nối u với v. Đường này không lặp đỉnh: Đoạn từ u lên l (không tính l) nằm trong nhánh con của l chứa u; đoạn từ v lên l (không tính l) nằm trong nhánh con chứa v. Hai nhánh này **khác nhau**, vì nếu chúng là một nhánh, gốc c của nhánh đó sẽ là một tổ tiên chung sâu hơn l, trái với định nghĩa l sâu nhất.

Vậy ta có một đường đi đơn nối u và v. Vì trong cây đường đi đơn là duy nhất, đây chính là đường đi từ u tới v, và nó đi qua l.

3. Bài toán HOHANG

BÀI TOÁN. HỌ HÀNG [HOHANG]

Gia phả của một dòng họ gồm n người đánh số từ 1 đến n; người số 1 là cụ tổ. Với mỗi v từ 2 đến n, cha của v là p[v]. Gia phả không quá **1000 đời**, tức là đường đi từ mỗi người lên cụ tổ có không quá 1000 cạnh. Có q câu hỏi, mỗi câu hỏi gồm hai người u, v; hãy cho biết tổ tiên chung gần nhất của họ.

Dữ liệu vào: Dòng đầu gồm hai số n, q ($1 \le n \le 2 \cdot 10^5$, $1 \le q \le 5 \cdot 10^4$). Dòng thứ hai gồm n − 1 số p[2], p[3], ..., p[n]. Mỗi dòng trong q dòng tiếp theo gồm hai số u, v ($1 \le u, v \le n$).

Kết quả: Với mỗi câu hỏi, in ra LCA(u, v) trên một dòng.

Dữ liệu vào	Kết quả	Giải thích
14 5 1 1 1 2 2 3 4 4 6 6 7 9 10 14 11 5 14 12 13 6 14 8 8	6 2 1 6 8	Gia phả ở Hình 1.1. Năm câu hỏi chính là các ví dụ ở Hình 1.3 và Hình 1.4.
4 2 3 4 1 2 4 3 2	4 3	Gia phả là đường thẳng 1, 4, 3, 2. Chú ý cha có thể mang số hiệu lớn hơn con.

Ví dụ thứ hai nhắc em một chi tiết dễ bị bỏ qua: **Không có gì đảm bảo p[v] < v.** Vì thế khi cần tính độ sâu, ta không thể duyệt v từ 2 đến n rồi gán dep[v] = dep[p[v]] + 1 (lúc đó dep[p[v]] có thể chưa được tính).

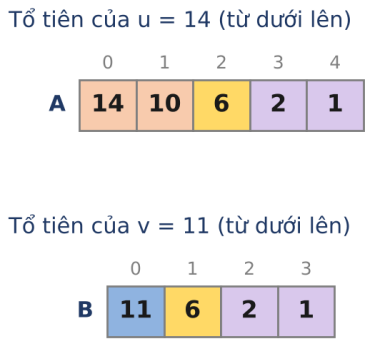
4. Ý tưởng ngây thơ: Liệt kê rồi so từng cặp

Cách làm đầu tiên ai cũng nghĩ ra là bắt chước định nghĩa:

1. Lập danh sách A gồm các tổ tiên của u theo thứ tự từ dưới lên: u, p[u], p[p[u]], ..., 1.
2. Lập danh sách B gồm các tổ tiên của v theo cùng cách.

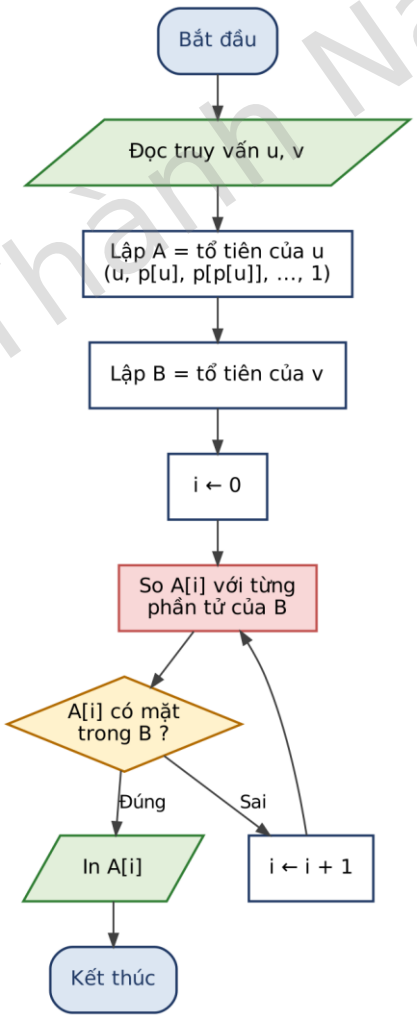
Lê Thành Nam

3. Duyệt A từ đầu (từ phần tử sâu nhất). Phần tử đầu tiên của A có mặt trong B chính là tổ tiên chung sâu nhất.



Hình 1.5. *Hai danh sách tổ tiên khi u = 14, v = 11. Phần tử đầu tiên của A có mặt trong B là A[2] = 6.*

Bước 3 đúng vì A được xếp theo độ sâu giảm dần: Phần tử đầu tiên thuộc B cũng là tổ tiên chung sâu nhất. Sơ đồ khối như sau.



Hình 1.6. *Sơ đồ khối của cách ngây thơ. Ô tô đỏ là chỗ tốn thời gian nhất.*

Lê Thành Nam

Code ý chính: Cách ngây thơ: Liệt kê rồi so từng cặp

```
vector<int> A, B;
for (int x = u; x != 0; x = p[x]) A.push_back(x);
for (int x = v; x != 0; x = p[x]) B.push_back(x);
int ans = 0;
for (int i = 0; i < (int)A.size() && ans == 0; i++)
    for (int j = 0; j < (int)B.size(); j++)
        if (A[i] == B[j]) {
            ans = A[i];
            break;
        }
```

Chạy tay với u = 14, v = 11: A = [14, 10, 6, 2, 1], B = [11, 6, 2, 1]. Với A[0] = 14, so với cả 4 phần tử của B, không trùng. Với A[1] = 10, cũng không trùng. Với A[2] = 6, so với B[0] = 11 rồi B[1] = 6: Trùng. Kết quả 6, sau tất cả 4 + 4 + 2 = 10 phép so sánh.

ĐẾM SỐ PHÉP TÍNH

Mỗi danh sách có không quá h + 1 phần tử, với h là chiều cao cây (ở bài này $h \le 1000$).

- Lập hai danh sách: Khoảng 2h bước.
- So từng cặp: Trường hợp xấu nhất, LCA là gốc (phần tử cuối của A), ta phải so gần như mọi cặp, khoảng $h \cdot h = h^2$ phép.

Vậy mỗi câu hỏi tốn **$O(h^2)$**. Với h = 1000 và $q = 5 \cdot 10^4$ câu hỏi: $5 \cdot 10^4 \cdot 10^6 = 5 \cdot 10^{10}$ phép, gấp năm trăm lần mức 10^8 phép mỗi giây. Chương trình chạy gần một phút.

BẾ TẮC

Chỗ lãng phí nằm ở bước so từng cặp: Với mỗi phần tử của A, ta lại dò lại **từ đầu** cả danh sách B. Cùng một câu hỏi "x có nằm trong B không?" được trả lời bằng cách duyệt tuần tự hàng nghìn lần.

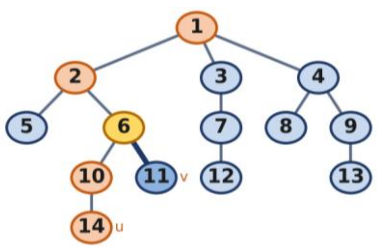
Câu hỏi: Có cách nào để kiểm tra "x có phải tổ tiên của v không" chỉ trong một bước? Và tốt hơn nữa: Có cách nào để không cần lập danh sách nào cả?

Lê Thành Nam

5. Phá bế tắc

Bước 1 Đánh dấu thay cho so từng cặp

Thay vì lưu B rồi dò, ta **đánh dấu** các tổ tiên của u vào một mảng mark[]. Sau đó đi lên từ v: Đỉnh đầu tiên đã được đánh dấu chính là LCA. Kiểm tra "x có được đánh dấu không" chỉ tốn một phép truy cập mảng.



Hình 1.7. Đánh dấu 14, 10, 6, 2, 1 (tô cam). Từ v = 11 đi lên một bước đã gặp 6, đỉnh đã được đánh dấu.

Vấn đề còn lại: Sau mỗi câu hỏi, phải **xóa dấu** để câu hỏi sau dùng lại mảng. Nếu xóa cả mảng bằng một vòng lặp n phần tử thì mỗi câu hỏi tốn O(n), quá chậm. Mẹo quen thuộc là dùng **"dấu thời gian"**: Thay vì đánh dấu bằng true, ta ghi số thứ tự của câu hỏi. Đỉnh x được coi là đã đánh dấu ở câu hỏi thứ t khi mark[x] == t. Sang câu hỏi t + 1, mọi dấu cũ tự động mất hiệu lực mà không cần xóa.

Code ý chính: Đánh dấu bằng dấu thời gian, câu hỏi thứ t

```
for (int x = u; x != 0; x = p[x]) mark[x] = t;
int x = v;
while (mark[x] != t) x = p[x];
```

Vòng while chắc chắn dừng, vì gốc 1 luôn được đánh dấu. Mỗi câu hỏi tốn O(h) thời gian, tổng cộng $O(q \cdot h) = 5 \cdot 10^4 \cdot 2 \cdot 10^3 = 10^8$ phép đơn giản: Đủ nhanh. Nhưng ta còn làm tốt hơn được, theo một cách **không cần mảng đánh dấu** và mở đường cho các chương sau.

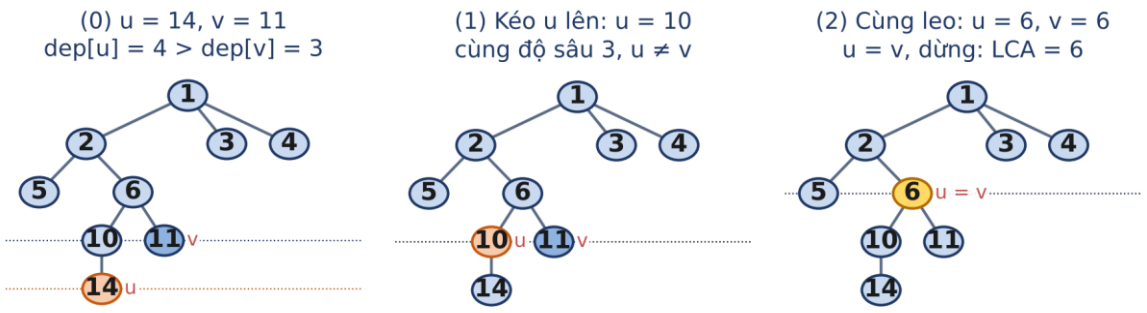
Bước 2 Quan sát: Hai đỉnh cùng độ sâu thì cùng leo sẽ gặp nhau tại LCA

Giả sử u và v có **cùng độ sâu**. Cho cả hai cùng đi lên cha, mỗi lần một bước. Vì luôn cùng độ sâu, hai "người leo" luôn ở cùng một tầng. Từ lúc chúng tới l = LCA(u, v), chúng ở cùng một đỉnh. Trước lúc đó, chúng ở hai đỉnh khác nhau (vì nếu đã trùng ở một đỉnh sâu hơn l thì đỉnh ấy là tổ tiên chung sâu hơn l, vô lí). Vậy:

> **CÔNG THỨC: CÙNG LEO**
> Nếu dep[u] = dep[v] thì lặp "u ← p[u], v ← p[v]" cho tới khi u = v. Đỉnh gặp nhau chính là LCA của u và v ban đầu.

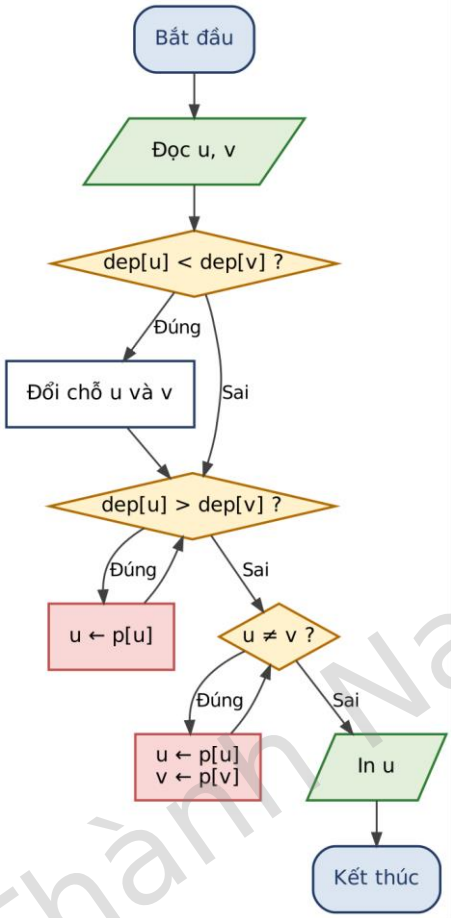
Bước 3 Cân độ sâu trước khi cùng leo

Nếu u và v không cùng độ sâu, chẳng hạn dep[u] > dep[v], ta **kéo u lên** dep[u] − dep[v] bước để đưa về cùng tầng với v. Việc kéo này không làm thay đổi đáp án: Đỉnh u' mới là tổ tiên của u nằm ở độ sâu dep[v]; LCA(u, v) có độ sâu không vượt quá dep[v], nên nó cũng là tổ tiên của u', và LCA(u', v) = LCA(u, v).



Hình 1.8. Ba bước tìm LCA(14, 11). (0) dep[14] = 4 lớn hơn dep[11] = 3. (1) Kéo u lên một bước thành 10, hai đỉnh cùng tầng. (2) Cùng leo một bước, cả hai tới 6: dừng.

Lê Thanh Nam



Hình 1.9. Sơ đồ khối của cách cân độ sâu rồi cùng leo.

Code ý chính: Cân độ sâu rồi cùng leo

```
if (dep[u] < dep[v]) swap(u, v);
while (dep[u] > dep[v]) u = p[u];
while (u != v) {
    u = p[u];
    v = p[v];
}
```

Bước 4 Chạy tay

Bảng dưới chạy tay ba câu hỏi trên cây ví dụ. Mỗi dòng là trạng thái (u, v) sau một bước.

Câu hỏi	Các trạng thái (u, v)	Giải thích
(14, 11)	(14, 11) → (10, 11) → (6, 6)	Kéo 14 lên 1 bước, cùng leo 1 bước.
(5, 14)	(14, 5) → (10, 5) → (6, 5) → (2, 2)	Đổi chỗ, kéo lên 2 bước, cùng leo 1 bước.
(12, 13)	(12, 13) → (7, 9) → (3, 4) → (1, 1)	Cùng độ sâu 3, cùng leo 3 bước tới gốc.
(6, 14)	(14, 6) → (10, 6) → (6, 6)	Sau khi kéo lên đã trùng: 6 là tổ tiên của 14.

Lê Thành Nam

Câu hỏi cuối cho thấy một trường hợp đặc biệt được xử lý tự nhiên: Khi u là tổ tiên của v, sau bước cân độ sâu hai đỉnh đã trùng nhau và vòng cùng leo không chạy lần nào.

Bước 5 Tính độ sâu khi cha có thể mang số lớn hơn con

Ta cần dep[] cho mọi đỉnh. Vì không có p[v] < v, ta duyệt theo chiều rộng (BFS) từ gốc: Lưu các con của mỗi đỉnh vào danh sách g[], đưa gốc vào hàng đợi; lấy ra đỉnh u nào thì gán dep cho các con của u và đưa chúng vào hàng đợi. Cha luôn được lấy ra trước con, nên dep[u] luôn có sẵn khi tính dep[v] = dep[u] + 1.

Code ý chính: Tính độ sâu bằng BFS, không cần đệ quy

```
vector<int> bfs;
bfs.push_back(1);
for (int i = 0; i < (int)bfs.size(); i++) {
    int u = bfs[i];
    for (int v : g[u]) {
        dep[v] = dep[u] + 1;
        bfs.push back(v);
    }
}
```

Mảng bfs vừa là hàng đợi (đọc bằng chỉ số i), vừa giữ lại toàn bộ thứ tự duyệt. Ta sẽ dùng lại thứ tự này nhiều lần ở các bài sau.

VÌ SAO ĐÚNG?: VÌ SAO CÂN ĐỘ SÂU RỒI CÙNG LEO CHO ĐÚNG LCA?

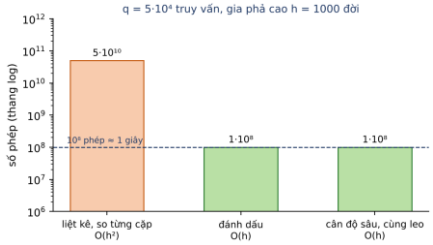
Gọi l = LCA(u, v) và giả sử dep[u] ≥ dep[v].

- Sau khi kéo u lên: u' là tổ tiên của u ở độ sâu dep[v]. Mọi tổ tiên chung của u và v có độ sâu không quá dep[v], nên nằm trên đường từ u' lên gốc. Ngược lại mọi tổ tiên của u' đều là tổ tiên của u. Do đó tập tổ tiên chung của (u', v) đúng bằng của (u, v), và LCA không đổi.
- Khi cùng leo: Sau k bước, u' và v trở thành tổ tiên ở độ sâu dep[v] − k của chúng. Chúng trùng nhau khi và chỉ khi tổ tiên ở tầng đó là chung. Tầng sâu nhất có tổ tiên chung chính là tầng của l, nên lần đầu tiên trùng nhau xảy ra đúng tại l.

ĐẾM SỐ PHÉP TÍNH

- Tính độ sâu bằng BFS: Mỗi đỉnh vào hàng đợi một lần, mỗi cạnh được xét một lần, tổng O(n).
- Mỗi câu hỏi: Vòng kéo lên chạy dep[u] − dep[v] ≤ h lần, vòng cùng leo chạy dep[v] − dep[l] ≤ h lần. Vậy mỗi câu hỏi tốn O(h), trong đó h là chiều cao cây.
- Tổng: O(n + q · h). Với n = 2·105, q = 5·104, h = 1000: Khoảng 2·105 + 108 phép đơn giản. Chương trình chạy khoảng 0,4 giây trên dữ liệu xấu nhất.

Lê Thanh Nam



Hình 1.10. *Số phép tính của ba cách làm khi $q = 5 \cdot 10^4$ và $h = 1000$ (thang logarit).*

6. Chương trình hoàn chỉnh

Phần đọc tệp được viết theo kiểu "có tệp thì đọc tệp": Khi chấm bằng Themis với tệp HOHANG.INP, chương trình đọc và ghi tệp; khi nộp lên trang chấm trực tuyến (dữ liệu vào từ bàn phím), điều kiện fopen sai nên chương trình đọc chuẩn như bình thường. Chương trình viết theo chuẩn C++14 và chỉ dùng các thư viện cần thiết thay cho bits/stdc++.h, để biên dịch được với mọi phiên bản Themis.

HOHANG.CPP: Cân độ sâu rồi cùng leo, O(n + q·h)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("HOHANG.INP", "r")) {
        freopen("HOHANG.INP", "r", stdin);
        freopen("HOHANG.OUT", "w", stdout);
    }
    int n, q;
    cin >> n >> q;
    vector<int> p(n + 1, 0), dep(n + 1, 0);
    vector<vector<int>> g(n + 1);
    for (int v = 2; v <= n; v++) {
        cin >> p[v];
        g[p[v]].push_back(v);
    }
    vector<int> bfs;
    bfs.push_back(1);
    for (int i = 0; i < (int)bfs.size(); i++) {
        int u = bfs[i];
        for (int v : g[u]) {
            dep[v] = dep[u] + 1;
            bfs.push_back(v);
        }
    }
    while (q--) {
        int u, v;
        cin >> u >> v;
        if (dep[u] < dep[v]) swap(u, v);
        while (dep[u] > dep[v]) u = p[u];
        while (u != v) {
            u = p[u];
            v = p[v];
        }
        cout << u << '\n';
    }
    return 0;
}
```

Lê Thành Nam

Vài chi tiết đáng chú ý:

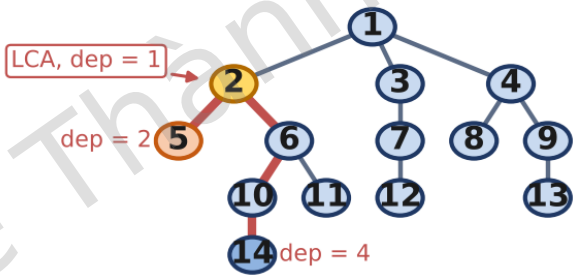
- Mảng p[] được giữ nguyên để leo cây; danh sách g[] chỉ lưu các con, nên BFS không cần kiểm tra "có phải cha không".
- Phép swap đảm bảo u luôn là đỉnh sâu hơn, nhờ đó chỉ cần một vòng kéo lên.
- Đọc ghi nhanh bằng ios_base::sync_with_stdio(false) và cin.tie(nullptr); in '\n' thay cho endl để không xả bộ đệm sau mỗi dòng.
- Chương trình đã được đối chiếu với lời giải vét cạn trên 300 bộ dữ liệu ngẫu nhiên gồm cây ngẫu nhiên, đường thẳng, hình sao, hình "con sâu", với số hiệu đỉnh bị xáo trộn.

7. Mở rộng

Mở rộng 1: Khoảng cách giữa hai đỉnh

Theo tính chất 3, đường đi từ u tới v gồm đoạn từ u lên $l = \text{LCA}(u, v)$ dài $\text{dep}[u] - \text{dep}[l]$ cạnh, và đoạn từ l xuống v dài $\text{dep}[v] - \text{dep}[l]$ cạnh. Vậy:

> **CÔNG THỨC KHOẢNG CÁCH**
>
> $$d(u, v) = \text{dep}[u] + \text{dep}[v] - 2 \cdot \text{dep}[\text{LCA}(u, v)]$$



Hình 1.11. *$d(5, 14) = 2 + 4 - 2 \cdot 1 = 4$: Đường đi 5, 2, 6, 10, 14 có đúng 4 cạnh.*

Hai số $\text{dep}[u] - \text{dep}[l]$ và $\text{dep}[v] - \text{dep}[l]$ còn cho biết "quan hệ họ hàng": Với An (14) và Bình (11), hai số đó là 2 và 1, nghĩa là ông nội của An là bố của Bình. Bài tập 1.1 (BACHOHANG) khai thác đúng ý này.

Mở rộng 2: LCA của cả một nhóm người

Theo tính chất 4, $\text{LCA}(a_1, a_2, ..., a_k)$ tính được bằng cách gộp dần: $l = a_1$, rồi $l = \text{LCA}(l, a_2)$, $l = \text{LCA}(l, a_3)$, ... Mỗi bước tốn $O(h)$, cả nhóm tốn $O(k \cdot h)$. Bài tập 1.2 (CUOCHOP).

Mở rộng 3: Điểm hẹn của ba người

Ba người ở a, b, c muốn gặp nhau tại một đỉnh x sao cho tổng quãng đường ba người phải đi là nhỏ nhất. Có thể chứng minh x chính là đỉnh **sâu nhất** trong ba đỉnh LCA(a, b), LCA(b, c), LCA(c, a) (hai trong ba đỉnh này luôn trùng nhau), và tổng quãng đường bằng (d(a, b) + d(b, c) + d(c, a)) / 2. Bài tập 1.3 (DIEMHEN) hướng dẫn chi tiết.

Lê Thành Nam

Mở rộng 4: Khi gia phả có 200 nghìn đời

Nếu bỏ điều kiện "không quá 1000 đời", cây có thể là một **đường thẳng** $2 \cdot 10^5$ đỉnh. Khi đó $h \approx 2 \cdot 10^5$, mỗi câu hỏi giữa hai đỉnh ở đáy cây và gần gốc tốn khoảng $2 \cdot 10^5$ bước, tổng $5 \cdot 10^4 \cdot 2 \cdot 10^5 = 10^{10}$ phép: Quá chậm.



Hình 1.12. Cây đường thẳng: chiều cao bằng n − 1, cách leo từng bước trở nên chậm.

BẾ TẮC CÒN LẠI

Leo cây từng bước một giống như leo cầu thang từng bậc. Khi cầu thang có hàng trăm nghìn bậc, ta cần một chiếc thang máy: Một cách "nhảy" nhiều bậc một lúc. Bài 3 sẽ dựng chiếc thang máy ấy bằng **kĩ thuật nhảy nhị phân**, đưa mỗi câu hỏi về O(log n) bước, bất kể cây cao bao nhiêu.

BẪY THƯỜNG GẶP

- Giả sử p[v] < v rồi tính dep[v] = dep[p[v]] + 1 theo thứ tự v tăng dần: Sai ngay ở ví dụ 2.
- Quên trường hợp u = v hoặc u là tổ tiên của v: Cách cân độ sâu xử lý được, nhưng nhiều cách viết tự chế thì không.
- Cùng leo khi hai đỉnh **chưa** cùng độ sâu: Hai người leo lệch tầng, có thể vượt qua LCA mà không gặp nhau, rồi cùng tới gốc và trả lời sai là 1.
- Xóa mảng đánh dấu bằng vòng lặp n phần tử sau mỗi câu hỏi: Mỗi câu hỏi thành O(n). Hãy dùng dấu thời gian.
- Dùng đệ quy để tính dep trên cây rất sâu: Có thể tràn ngăn xếp khi chấm trên Windows. BFS không có rủi ro này.
- Dùng endl khi in $5 \cdot 10^4$ dòng: Chương trình chậm đi nhiều lần.

Lê Thanh Nam

GHI NHỚ

- LCA(u, v) là tổ tiên chung sâu nhất của u và v; mỗi đỉnh là tổ tiên của chính nó.
- Đường đi từ u tới v luôn đi lên từ u tới LCA rồi đi xuống tới v; d(u, v) = dep[u] + dep[v] − 2·dep[LCA].
- Cân độ sâu rồi cùng leo: Kéo đỉnh sâu hơn lên cho bằng tầng, rồi cả hai cùng leo cho tới khi gặp nhau. Mỗi câu hỏi O(h).
- Tính độ sâu bằng BFS từ gốc, không giả sử cha có số hiệu nhỏ hơn con.
- Khi cây cao (h tới n), leo từng bước là quá chậm: Cần nhảy nhị phân (Bài 3).

Luyện tập

Bài	Tên	Mức độ	Gợi ý liên hệ
1.1	BACHOHANG	Cơ bản	Mở rộng 1: Từ LCA suy ra số đời từ mỗi người lên tổ tiên chung.
1.2	CUOCHOP	Trung bình	Mở rộng 2: LCA của cả nhóm bằng cách gộp dần.
1.3	DIEMHEN	Nâng cao	Mở rộng 3: Điểm hẹn của ba người và tổng quãng đường nhỏ nhất.

Tự kiểm tra nhanh

1. Trên cây ví dụ, LCA(11, 13) bằng bao nhiêu? *(Đáp số: 1.)*

2. Trên cây ví dụ, d(12, 14) bằng bao nhiêu? *(Đáp số: 3 + 4 − 2·0 = 7.)*

3. Nếu chọn gốc là đỉnh 6, LCA(5, 11) bằng bao nhiêu? *(Đáp số: 6, vì đường đi từ 5 tới 11 là 5, 2, 6, 11 và 6 là đỉnh gần gốc mới nhất trên đường đó.)*

4. Cây đường thẳng 1, 2, ..., n gốc 1. Cách cân độ sâu rồi cùng leo tốn bao nhiêu bước với câu hỏi (n, 1)? *(Đáp số: n − 1 bước kéo lên, 0 bước cùng leo.)*

Lê Thành Nam